

The modules for various forms of input fields will look like the ones seen below. As required, each variable will have its own collection of methods, details, and props.

- Text field is rendered by `<InputBox>`.
- Checkboxes are rendered by `<Checkbox>`.
- `<RadioButton>` renders radio buttons.

Every one of the components mentioned above will embrace user feedback and return it to the parent component. The meaning that is released would be saved in a local object variable called `formData`.

Simply put, we can:

- register building block components in components: {} key,
- Loop through the form's settings,
- Using the `:is` attribute, make each field with `<component>`.
- `v-model` can be used to save the emitted value in a local variable: `data-Form`

```
<template v-for="(field, key) in formFields">
  <Component
    v-model="formData[field.name]"
    :is="field.type === text ? InputBox : ''"
    :type="field.type"
  />
</template>
```

Copyright Attribution: Medium

Form with `<component>`

Since the form configuration and Vue components are distinct elements, we must first locate the corresponding component for each field before using `:is`.

We have alternatives:

1. To find and assign a matching component, use `v-if/v-else`, or
2. To delegate the related Vue component, use `switch` statements, or
3. Better still, delegate the corresponding part for each field using an object chart.

We may also prepare a whole field object using the... spread operator to amend the portion key into a simple field object in the third choice above.

This choice will also save us a ton of `ifs-and-else's`.